

# Builder

---

Pitavy Mathis

Priot Louise

Teixeira da Silva Samuel

# SOMMAIRE

---

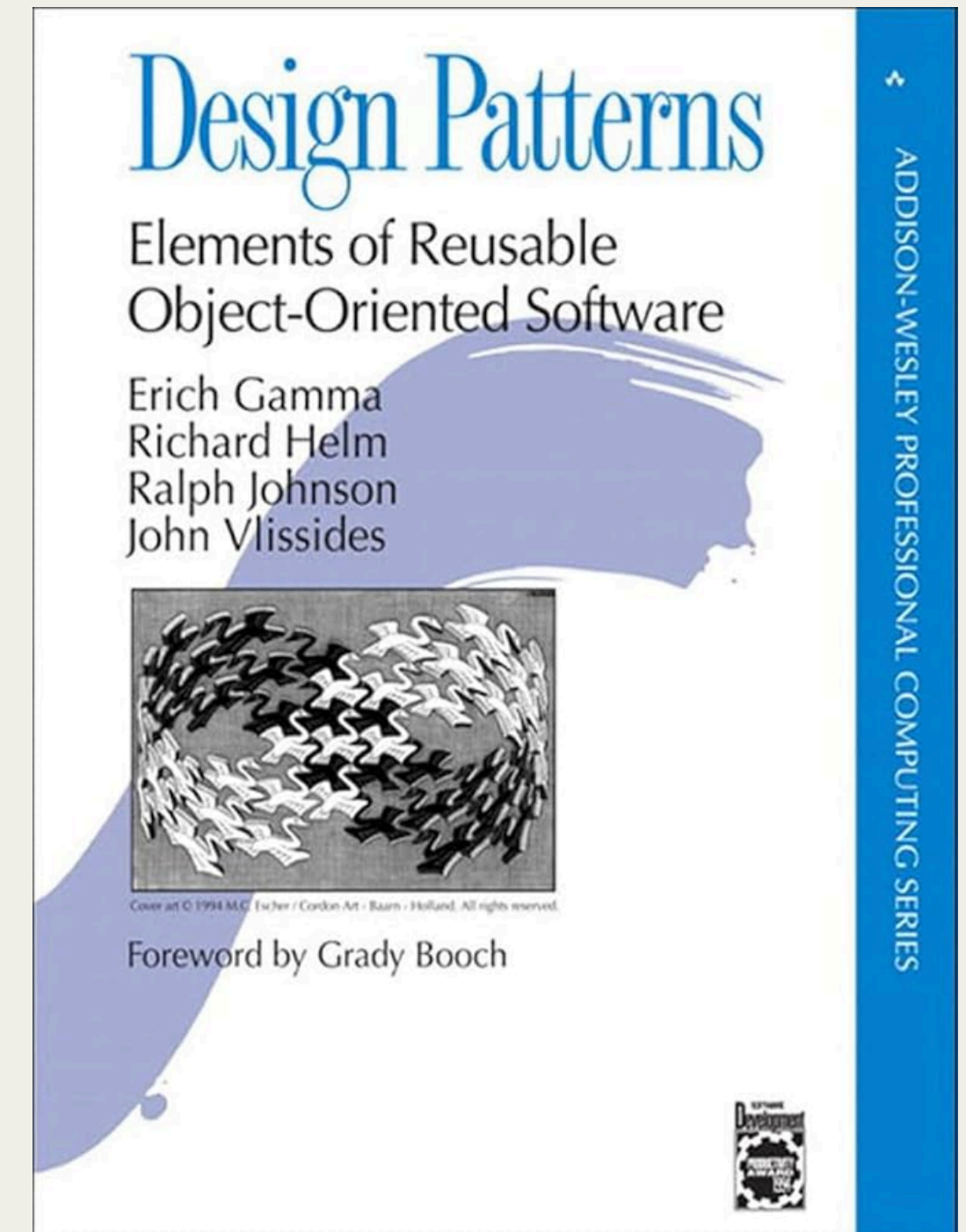
- Le design Partten
- Les 3 familles Pattern
- Identité du Pattern
- Exemple pour Pattern Builder
- Première modélisation
- Illustration du problème
- Solution
- UML générique

# LES DESIGN PATTERNS

---

- Définition: Les patrons de conception sont des solutions classiques à des problèmes récurrents de la conception de logiciels.

➔ Le pattern Builder



# LES 3 FAMILLES PATERN

---

- CRÉATION
- STRUCTURE
- COMPORTEMENT

# UN EXEMPLE POUR NOTRE PATTERN BUILDER

---

- Contexte : Développement d'un logiciel de gestion pour une maison d'édition.
- Besoin : Instancier des objets représentant des livres avec de nombreuses caractéristiques.
- Attributs identifiés :
  - titre
  - auteur
  - ISBN
  - résumé
  - maison d'édition
  - prix
  - jaspé
  - signet
  - rabat

# UNE PREMIÈRE MODÉLISATION

---

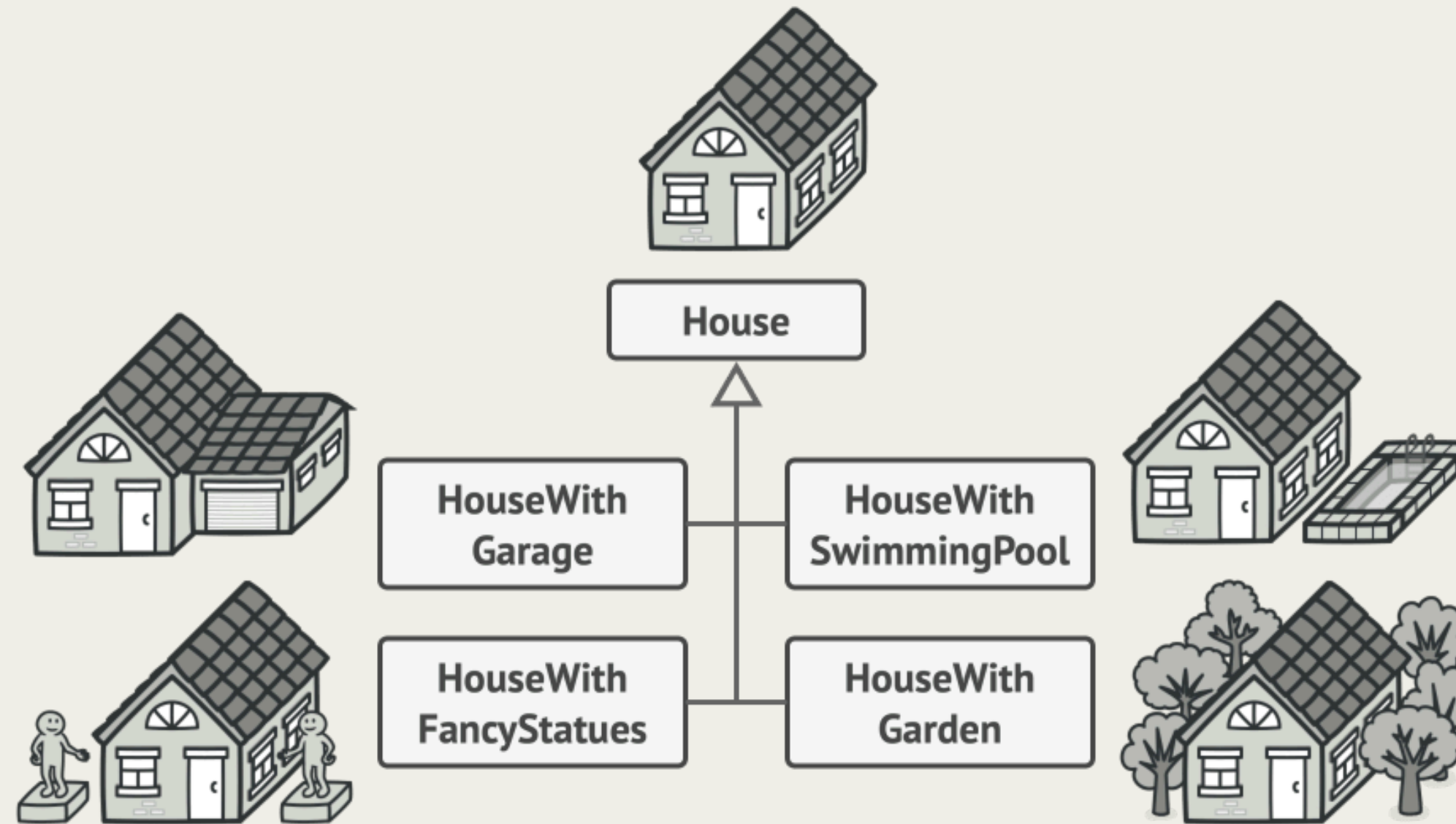
| C Livre                                                                                                                                                                                                                                                       |  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| <div><div>▣ titre: String</div><div>▣ auteur: String</div><div>▣ isbn: String</div><div>▣ resume: String</div><div>▣ prix: float</div><div>▣ editeur: String</div><div>▣ jaspague: boolean</div><div>▣ signet: boolean</div><div>▣ rabat: boolean</div></div> |  |
| <div><div>● Livre(titre: String, auteur: String, isbn: String, resume: String, prix: float, editeur: String, jaspague: boolean, signet: boolean, rabat: boolean)</div><div>● toString(): String</div></div>                                                   |  |



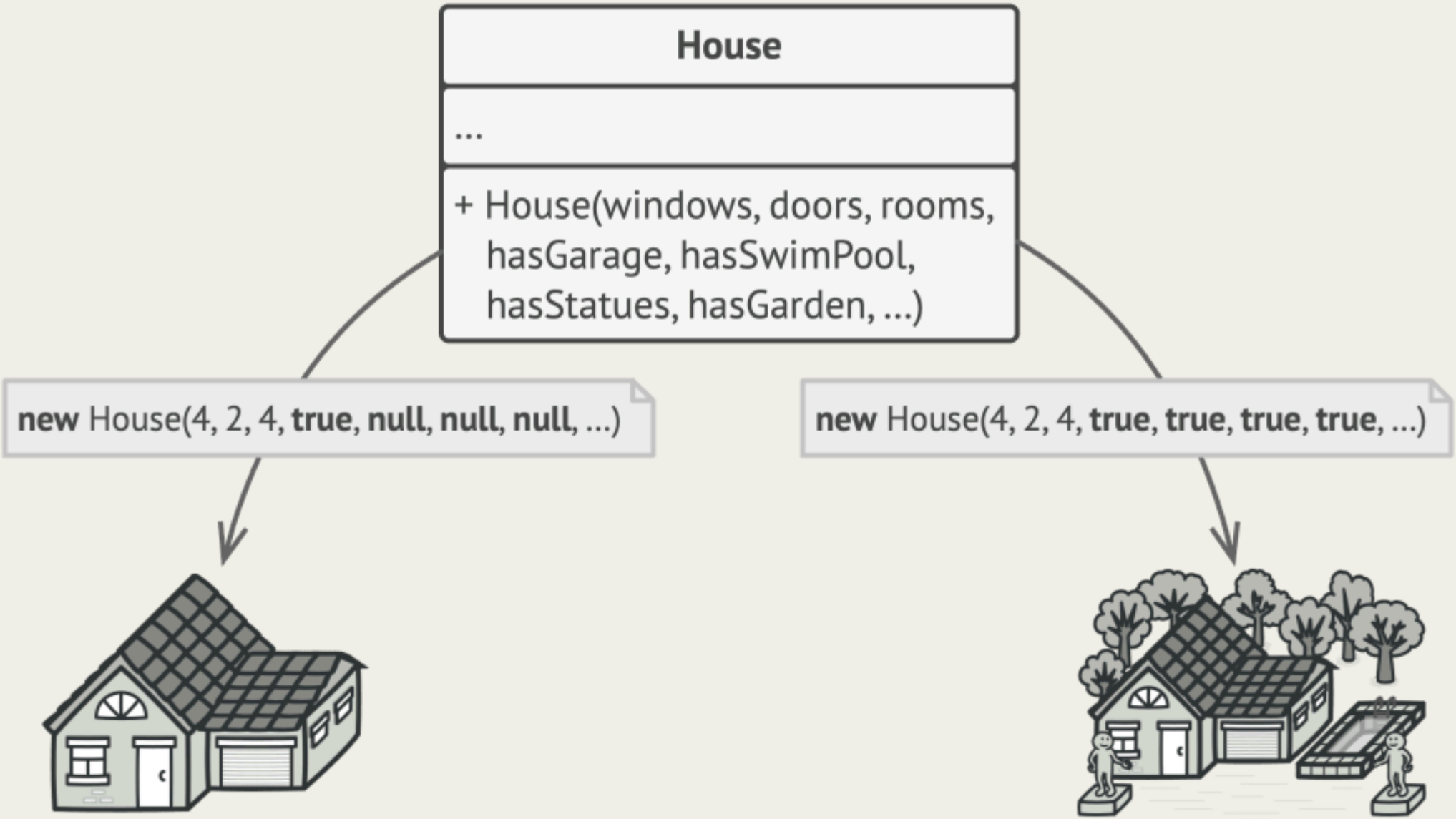
Anti-pattern du constructeur télescopique

# ILLUSTRATION DU PROBLEME

---

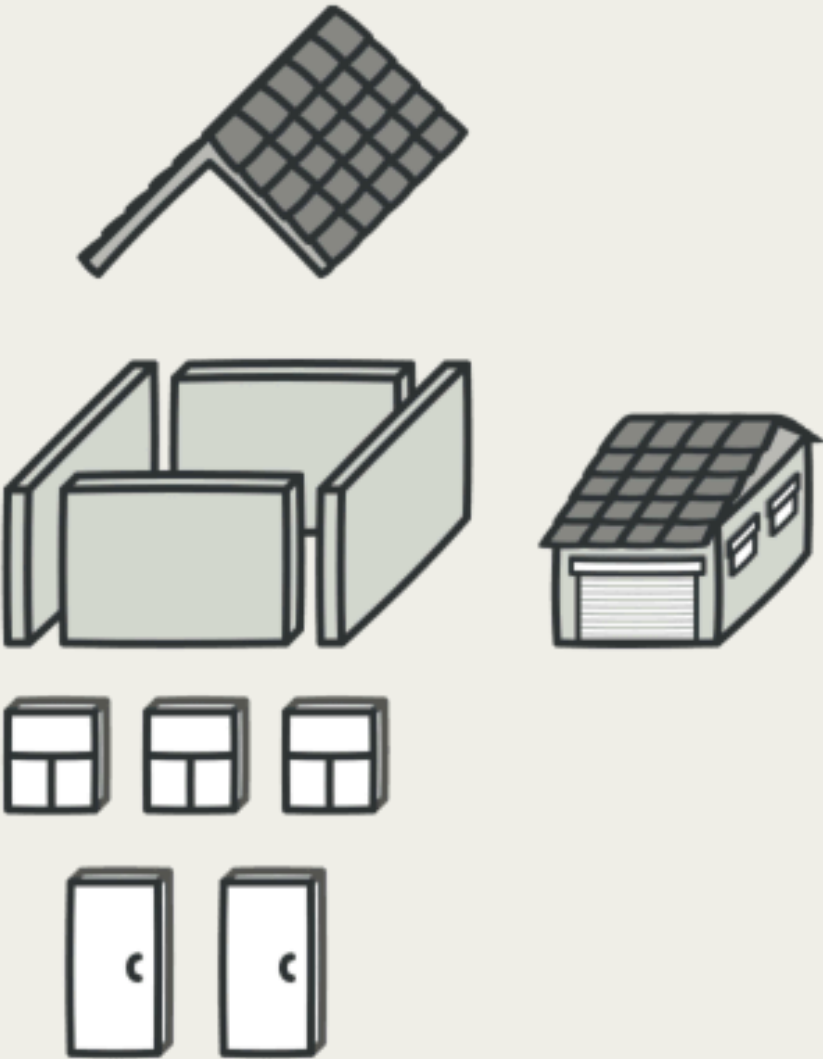
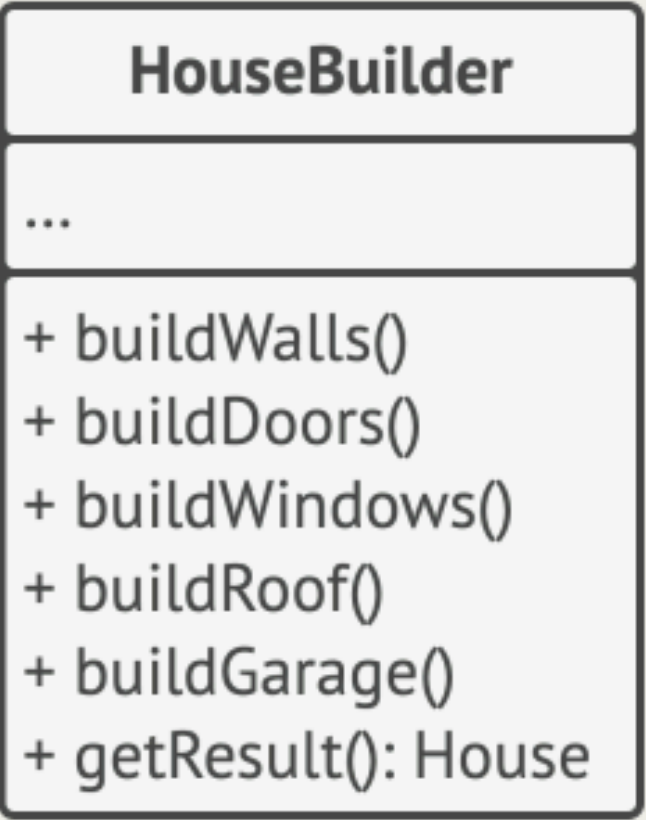


# ILLUSTRATION DU PROBLEME

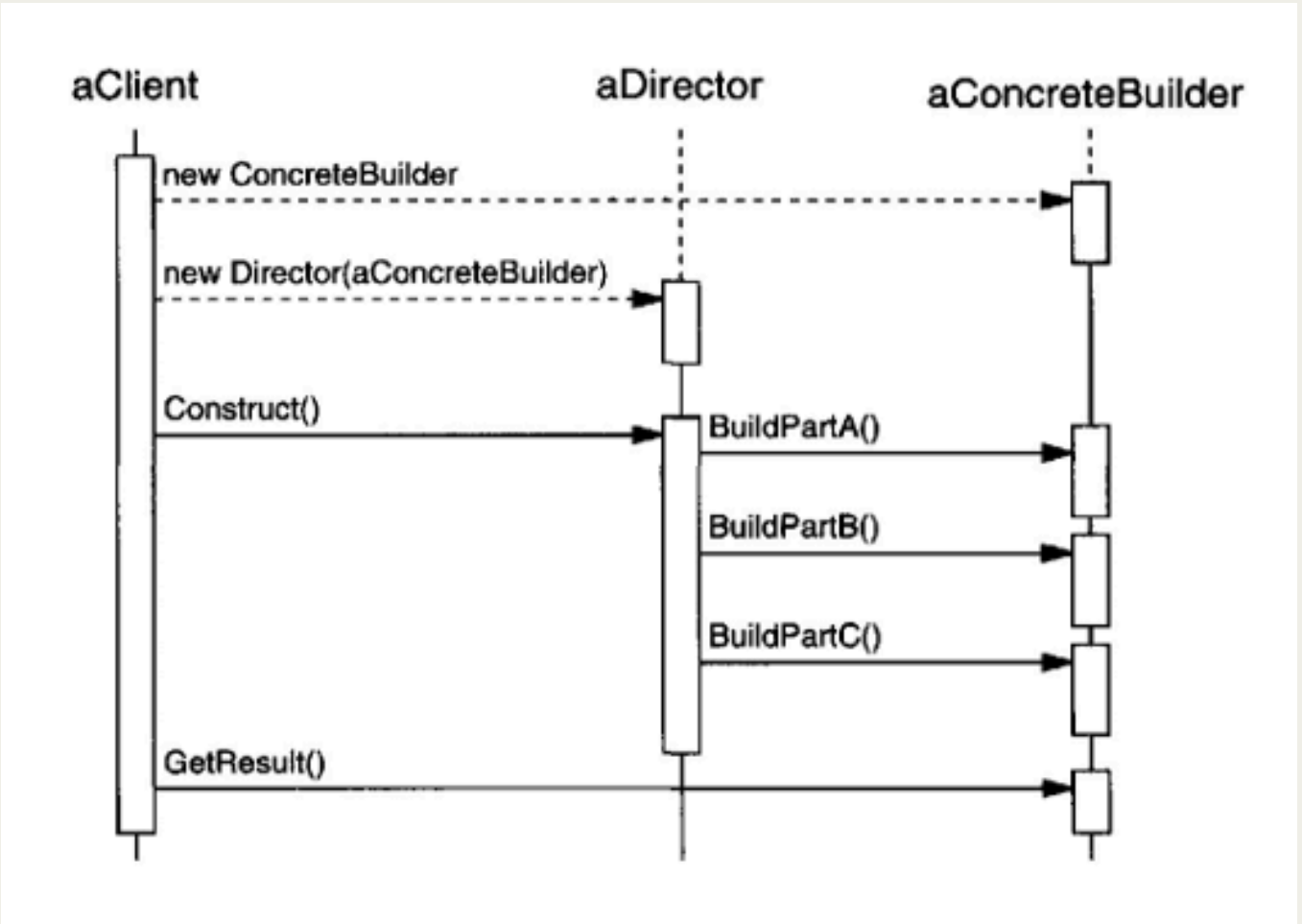
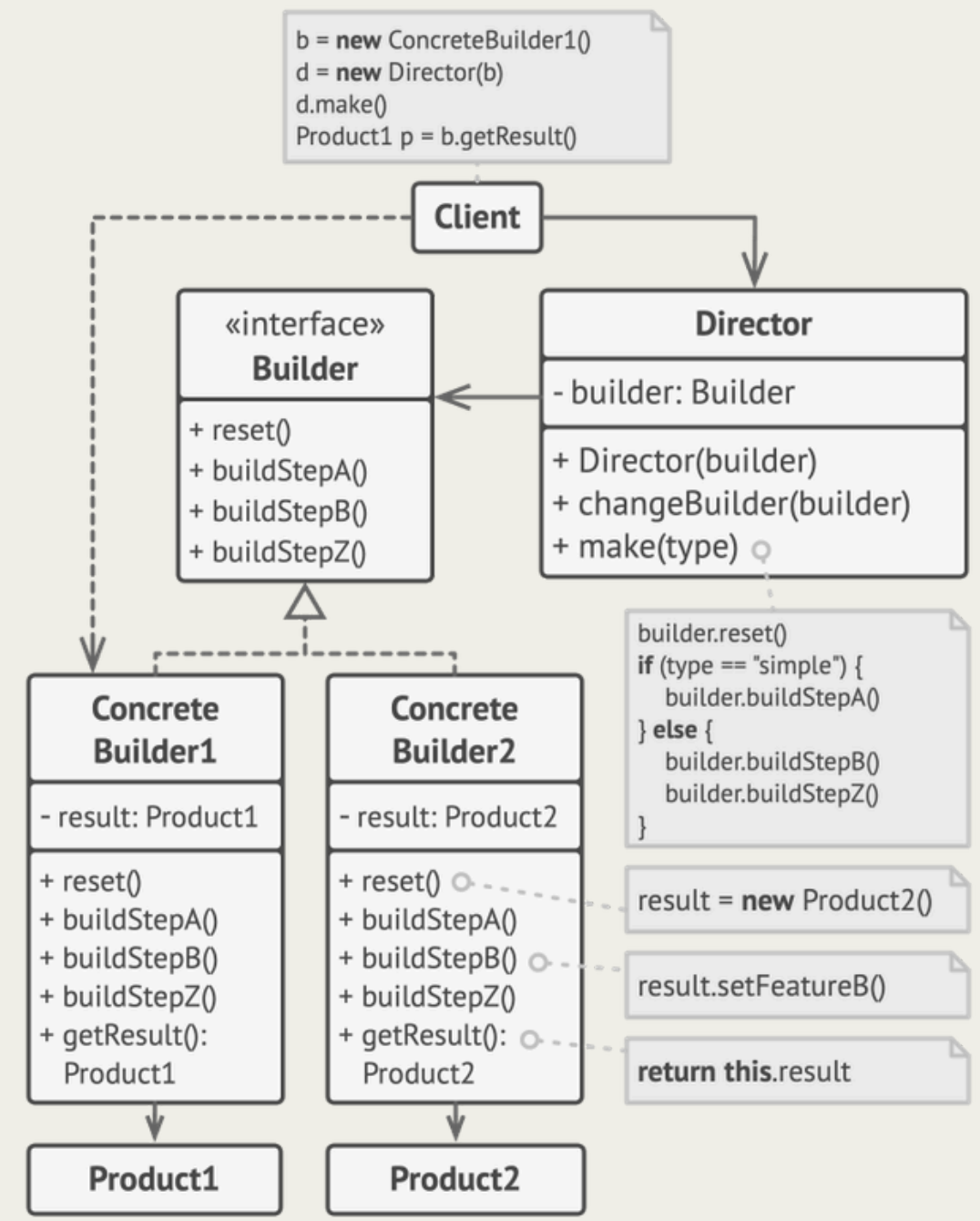


# SOLUTION

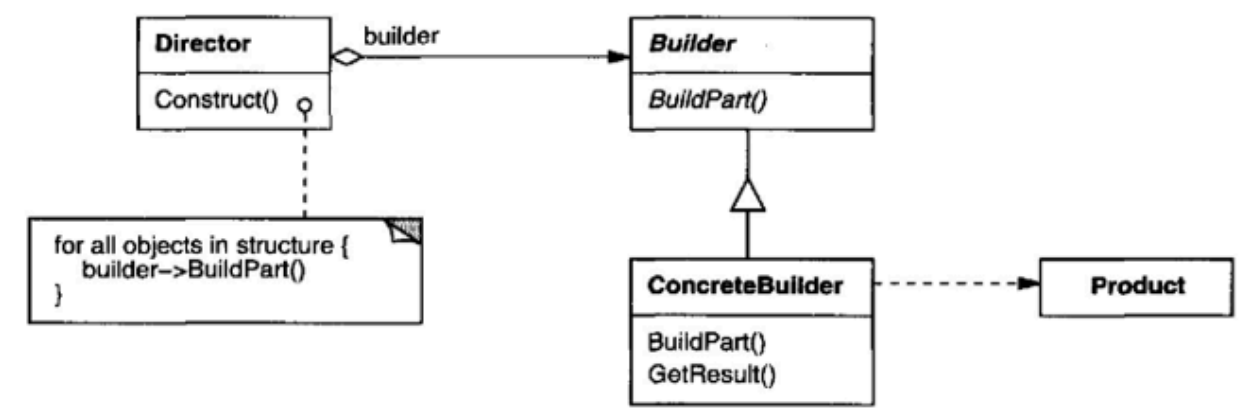
---



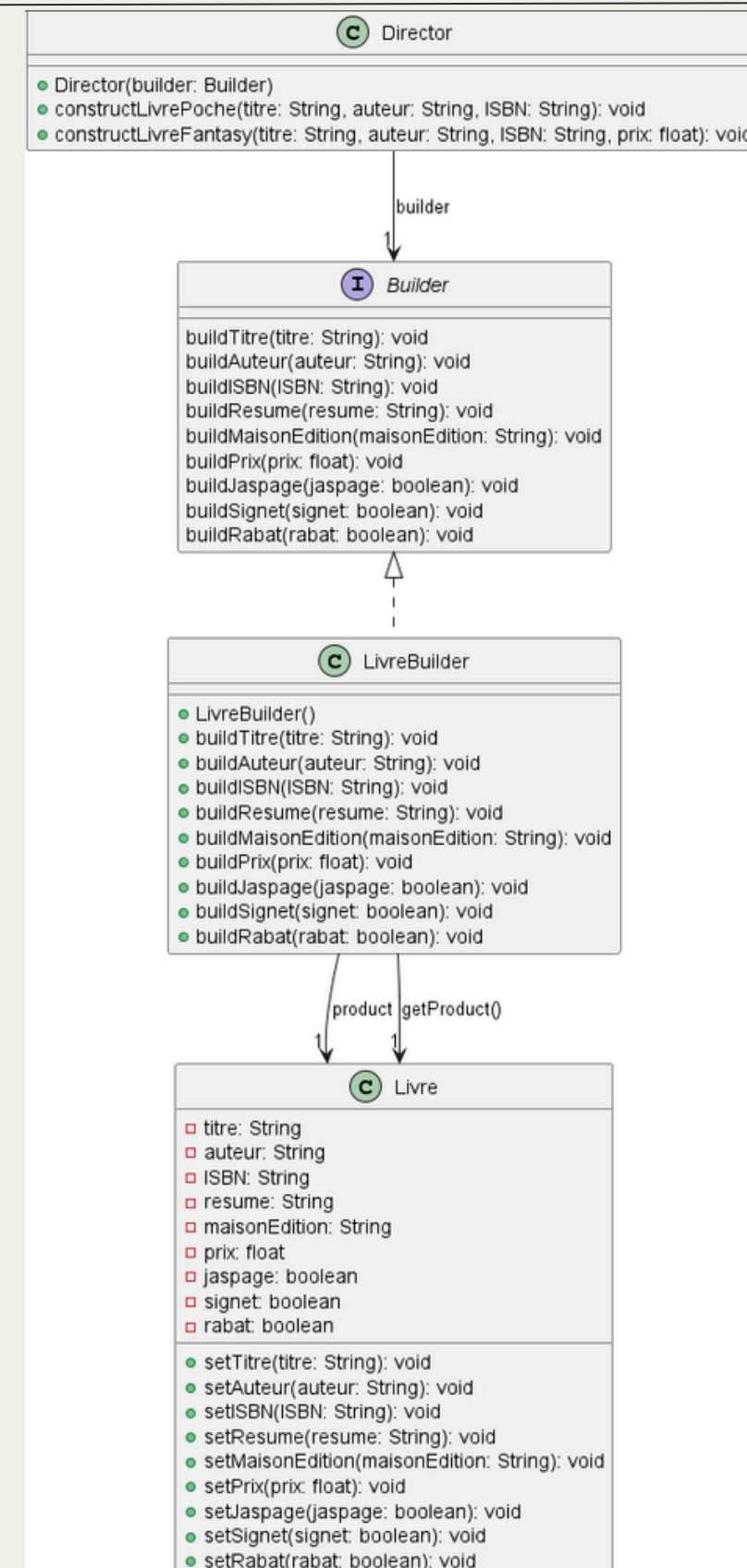
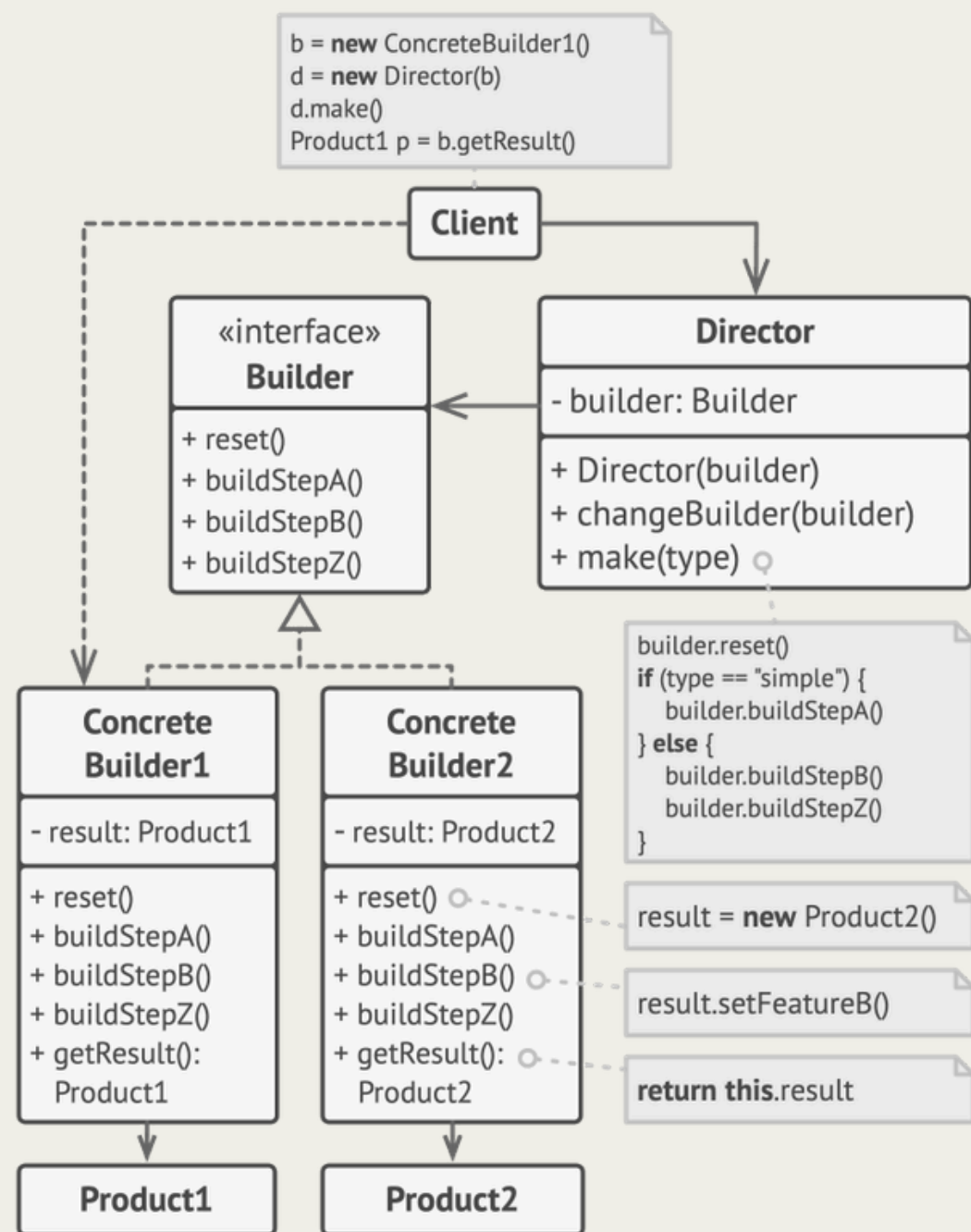
# UML GÉNÉRIQUE



## Structure



# UML GÉNÉRIQUE VS APPLICATION A NOTRE EX



# AVANT VS APRES

---

```
public static void main(String[] args) {  
    LivreMauvaiseModelisation designPattern = new LivreMauvaiseModelisation("Design Patterns",  
        "Erich Gamma\nRichard Helm\nRalph Johnson\nJohn Vlissides",  
        "0-201-63361-2",  
        "Design patterns, software engineering, object-oriented programming",  
        "Addison-Wesley",  
        22.77f, false, false, false);  
}
```

```
public static void main(String[] args) {  
    //Instancier le constructeur concret  
    LivreBuilder builder = new LivreBuilder();  
  
    //Instancier le directeur en lui passant le constructeur  
    Director director = new Director(builder);  
  
    //Le directeur construit le produit selon une recette precise  
    director.constructLivrePoche("Le Petit Prince", "Antoine de Saint-Exupéry", "2070612759");  
  
    //Recuperer le produit final directement depuis le constructeur  
    Livre monLivrePoche = builder.getProduct();  
  
    System.out.println(monLivrePoche);  
  
    //Le directeur construit le produit selon une recette precise  
    director.constructLivreFantasy("Nom Livre Fantasy", "Auteur Livre Fantasy", "ISBN", 34.99f);  
  
    //Recuperer le produit final directement depuis le constructeur  
    Livre monLivreFantasy = builder.getProduct();  
  
    System.out.println(monLivreFantasy);  
}
```

Le Petit Prince de Antoine de Saint-Exupéry, ISBN: 2070612759, resume: null, édité par Poche, prix: 8.5, jaspé: false, signet: false, rabat: false.  
Nom Livre Fantasy de Auteur Livre Fantasy, ISBN: ISBN, resume: null, édité par Poche, prix: 34.99, jaspé: true, signet: true, rabat: true.

# IDENTITÉ DU PATTERN

---

- Permet de construire progressivement un objet complexe.
- Sépare la construction de l'objet de sa représentation.
- Un même processus de construction peut produire différents résultats selon les paramètres choisis.

# PROBLÉMATIQUE ET INTENTION

---

- Sans le pattern Builder :
  - Multiplication des constructeurs.
  - Nombreuses valeurs null ou false.
  - Difficulté à identifier les paramètres et leur correspondance avec les attributs.
  - Code moins lisible et plus difficile à maintenir.
- Avec le pattern Builder :
  - Construction de l'objet étape par étape.
  - Choix libre des attributs à renseigner.
  - Possibilité de créer différentes combinaisons de paramètres.
  - Utilisation de Directors pour préconfigurer des constructions courantes.

Objectif : simplifier la création d'objets complexes tout en améliorant la lisibilité et la flexibilité du code.

# SOLID

- **S.R.P** : **S**ingle **R**esponsibility **P**rinciple
- **O.C.P** : **O**pen/**C**losed **P**rinciple
- **L.S.P** : **L**iskov **S**ubstitution **P**rinciple
- **I.S.P** : **I**nterface **S**egregation **P**rinciple
- **D.I.P** : **D**ependency **I**nversion **P**rinciple

## Design Principles and Design Patterns

Robert C. Martin  
www.objectmentor.com

What is software architecture? The answer is multitiered. At the highest level, there are the architecture patterns that define the overall shape and structure of software applications<sup>1</sup>. Down a level is the architecture that is specifically related to the purpose of the software application. Yet another level down resides the architecture of the modules and their interconnections. This is the domain of design patterns<sup>2</sup>, packages, components, and classes. It is this level that we will concern ourselves with in this chapter.

Our scope in this chapter is quite limited. There is much more to be said about the principles and patterns that are exposed here. Interested readers are referred to [Martin99].

### Architecture and Dependencies

What goes wrong with software? The design of many software applications begins as a vital image in the minds of its designers. At this stage it is clean, elegant, and compelling. It has a simple beauty that makes the designers and implementers itch to see it working. Some of these applications manage to maintain this purity of design through the initial development and into the first release.

But then something begins to happen. The software starts to rot. At first it isn't so bad. An ugly wart here, a clumsy hack there, but the beauty of the design still shows through. Yet, over time as the rotting continues, the ugly festering sores and boils accumulate until they dominate the design of the application. The program becomes a festering mass of code that the developers find increasingly hard to maintain. Eventu-

1. [Shaw96]

2. [GOF96]

# VERSION FLUENT

---

- ajoute la possibilité de chaîner les appels

```
ConcretePersonnageJeuBuilder guerrier = new ConcretePersonnageJeuBuilder();
guerrier.construireNom("Brom")
        .construiteRace("Nain")
        .construireClasse("Guerrier")
        .construireArme("Hache");
System.out.println(guerrier.resultat());
```

# LES AVANTAGES

---

- Meilleure lisibilité du code
- Gestion des paramètres optionnels
- Favorise de l'immutabilité

```
ConcretePersonnageJeuBuilder guerrier = new ConcretePersonnageJeuBuilder();
guerrier.construireNom("Brom")
        .construireRace("Nain")
        .construireClasse("Guerrier")
        .construireArme("Hache");
System.out.println(guerrier.resultat());
```

# LES INCONVÉNIENTS

---

- Code plus long
- Complexité
- Risque d'objets mal configurés

# TABLE DES PARTICIPANTS

---

| Rôle            | Responsabilité                                                                                   |
|-----------------|--------------------------------------------------------------------------------------------------|
| Builder         | Déclare l'interface des étapes de construction                                                   |
| ConcreteBuilder | Construit et assemble les parties, conserve la représentation et permet de récupérer le résultat |
| Director        | Orchestre les étapes en utilisant l'interface Builder                                            |
| Product         | Objet complexe en cours de construction, puis résultat final                                     |

# UN EXEMPLE POUR NOTRE PATTERN BUILDER

---

- Contexte : Développement d'un jeu RPG avec des donjons
- Besoin : Instancier des objets représentant des donjons avec de nombreuses caractéristiques.
- Attributs identifiés :
  - nom
  - difficulté
  - boss
  - étage
  - piège
  - coffre
  - monstre

# EXEMPLE : DONJON

```
package builder;

public class Donjon {
    private String nom;
    private String difficulte;
    private String boss;
    private int etage;
    private String piege;
    private String coffre;
    private String monstre;

    public Donjon(String nom, String difficulte, String boss,
        int etage, String piege, String coffre, String monstre) {
        this.nom = nom;
        this.difficulte = difficulte;
        this.boss = boss;
        this.etage = etage;
        this.piege = piege;
        this.coffre = coffre;
        this.monstre = monstre;
    }

    @Override
    public String toString() {
        return "Donjon [nom=" + nom + ", difficulte=" + difficulte
            + ", boss=" + boss + ", etage=" + etage + ", piege="
            + piege + ", coffre=" + coffre + ", monstre=" + monstre + "];"
    }
}
```

 Donjon

□ nom: String  
□ difficulte: String  
□ boss: String  
□ etage: int  
□ piege: String  
□ coffre: String  
□ monstre: String

● Donjon(nom: String, difficulte: String, boss: String, etage: int, piege: String, coffre: String, monstre: String)  
● toString(): String

# EXEMPLE : DONJON

```
public class ConcreteDonjonBuilder implements DonjonBuilder {
    private String nom;
    private String difficulte;
    private String boss;
    private int etage;
    private String piege;
    private String coffre;
    private String monstre;
    @Override
    public DonjonBuilder construireNom(String nom) {
        this.nom = nom;
        return this;
    }
    @Override
    public DonjonBuilder construireDifficulte(String difficulte) {
        this.difficulte = difficulte;
        return this;
    }
    @Override
    public DonjonBuilder construireBoss(String boss) {
        this.boss = boss;
        return this;
    }
    @Override
    public DonjonBuilder construireEtage(int etage) {}
    @Override
    public DonjonBuilder construirePiege(String piege) {}
    @Override
    public DonjonBuilder construireCoffre(String coffre) {}
    @Override
    public DonjonBuilder construireMonstre(String monstre) {}
    public Donjon resultat() {
        return new Donjon(nom, difficulte, boss, etage, piege, coffre, monstre);
    }
}
```

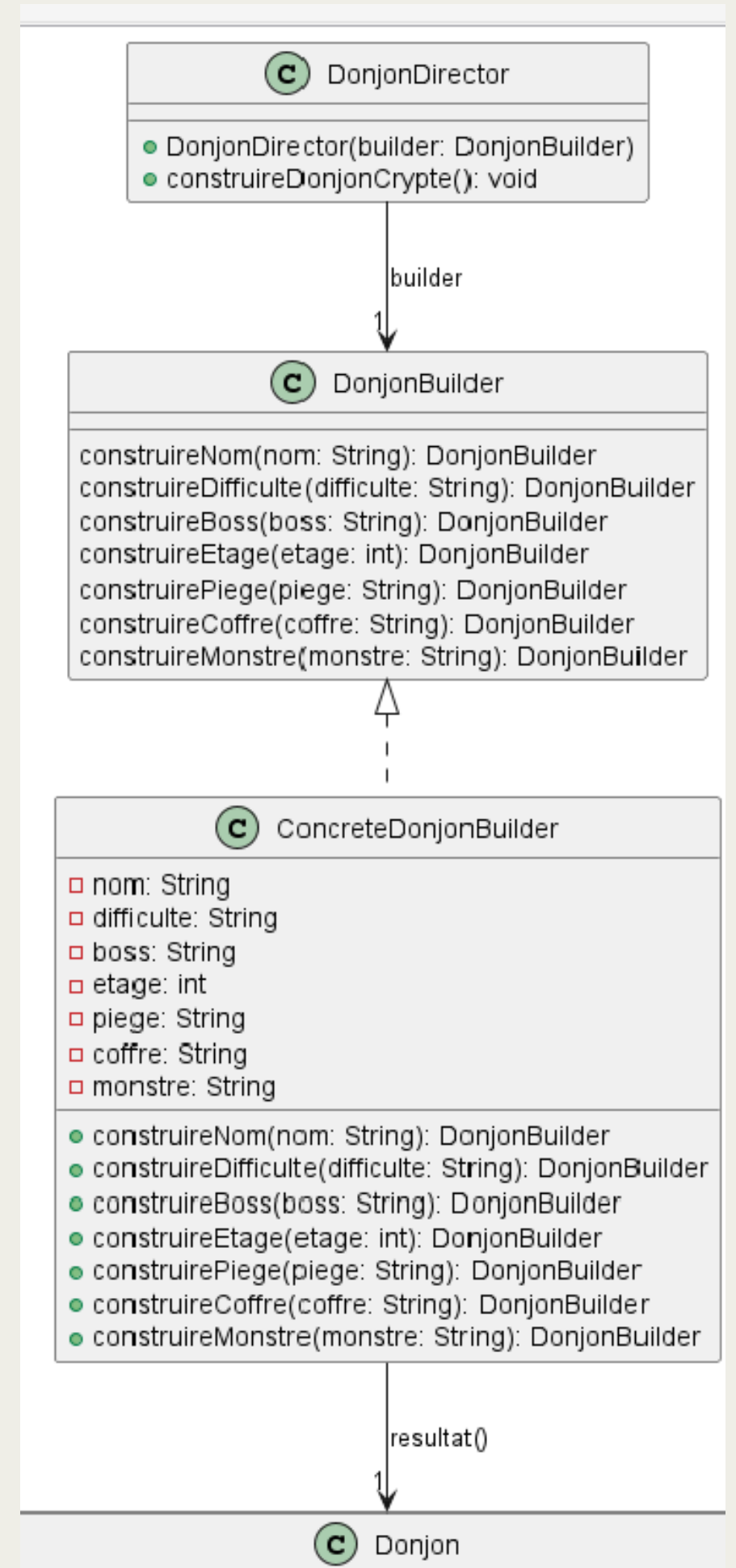
```
public class DonjonDirector {
    private final DonjonBuilder builder;

    public DonjonDirector(DonjonBuilder builder) {
        this.builder = builder;
    }

    public void construireDonjonCrypte() {
        builder.construireNom("Crypte")
            .construireDifficulte("Difficile")
            .construireBoss("Liche")
            .construireEtage(3)
            .construirePiege("Fosse")
            .construireCoffre("Coffre en or")
            .construireMonstre("Squelette");
    }
}
```

# EXEMPLE : DONJON

```
public class Main {  
  
    public static void main(String[] args) {  
        Donjon donjon = new Donjon("Crypte", "Difficile", "Liche", 3, "Fosse", null, null);  
        System.out.println(donjon);  
  
        ConcreteDonjonBuilder crypte = new ConcreteDonjonBuilder();  
        new DonjonDirector(crypte).construireDonjonCrypte();  
        System.out.println(crypte.resultat());  
  
        ConcreteDonjonBuilder donjonSimple = new ConcreteDonjonBuilder();  
        donjonSimple.construireNom("Grotte")  
            .construireDifficulte("Facile")  
            .construireEtage(1)  
            .construireMonstre("Gobelin");  
        System.out.println(donjonSimple.resultat());  
    }  
}
```



# LIVE CODING

---

<https://youtu.be/O8-RMoZ6NNU>

<https://glowit.io/dashboard>

# BIBLIOGRAPHIE

---

<https://refactoring.guru/fr/design-patterns/builder>

<https://yassinechraa.medium.com/le-pattern-builder-81a1f0875919>

<https://www.gofpattern.com/creational/patterns/builder-pattern.php>

<https://www.ionos.fr/digitalguide/sites-internet/developpement-web/stringbuilder-en-java/>

Design Patterns Elements of Reusable Object-Oriented Software.pdf

[https://fr.wikipedia.org/wiki/SOLID\\_%28informatique%28](https://fr.wikipedia.org/wiki/SOLID_%28informatique%28)

# Merci!

---

# IA GRAPHIE

---

- Louise :
  - Traduction d'extrait du PDF sur les Design Pattern
  - Proposition d'idées de valeurs pour les exemples
  - Correcteur orthographique
- Mathis:
  - explications supplémentaires
  - confirmation d'exemples
- Samuel :
  - Propositions d'idée de réponse pour certaines questions du QCM
  - Correcteur orthographique à certains moments